Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Título del Proyecto:

Sistema Integral de Monitoreo, Análisis y Respuesta de Seguridad a Nivel Kernel

PONDERACIÓN: 80 pts

🕐Tiempo estimado: 50 hrs

Índice

1. MARCO FORMATIVO..... 3
1.1. Valor..... 3
1.2. Competencia(s)..... 3
1.3. Objetivo SMART..... 3
2. Enunciado de la Práctica..... 4
3.1 Descripción del problema a resolver..... 4
3.2 Alcance de la práctica..... 4
3.3 Entregables..... 4
3. Material de apoyo..... 6
6. Recursos y herramientas a utilizar..... 7
7. Cronograma..... 7
8. Rúbrica de Calificación..... 7
8.1 Requisitos para optar a la calificación..... 7
8.2 Resumen de Puntuaciones..... 9
8.5 Comentarios Generales..... 9
Detalle de la Calificación..... 11

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Adaptabilidad	Explorar diferentes enfoques es indispensable para aumentar nuestra biblioteca personal de conocimiento. Una solución puede ser implementada de muchas formas, elegir la forma correcta es crítico para el futuro del sistema y por eso debe analizarse con cuidado cómo adaptarse a nuevos entornos.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Aplica principios básicos de ingeniería, ciencias de computación y sistemas de información y comunicación, en la formulación y resolución adecuada de problemas complejos.
Competencia Específica	Demuestra pensamiento crítico, actitud investigativa y rigor analítico en el planteamiento y la resolución de problemas complejos.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Diseñar e implementar un sistema integral de monitoreo y análisis de seguridad a nivel kernel, que permita recolectar métricas del sistema, analizar procesos y archivos mediante llamadas al sistema personalizadas, y detectar amenazas simuladas utilizando un mecanismo de firmas basado en hashes.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	El sistema se considerará completo cuando se implementan al menos 5 métricas del sistema, un mecanismo funcional de detección basado en una blacklist de al menos 8 firmas de amenazas simuladas, un sistema de alertas con clasificación de severidad (LOW, MEDIUM, HIGH), y un dashboard que visualice correctamente la información en tiempo real.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Se logrará mediante la implementación de syscalls en el kernel de Linux para la recolección de métricas y análisis de archivos, el desarrollo de un programa en espacio de usuario en C que procese la información y la envíe a un backend, y la construcción de un dashboard web que

		permita la visualización y gestión del sistema, incluyendo autenticación mediante PAM.
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Para comprender el funcionamiento interno de los sistemas operativos, el flujo de comunicación entre kernel y espacio de usuario, y la aplicación de conceptos de ciberseguridad como detección basada en firmas, monitoreo de recursos y control de acceso, simulando el comportamiento de herramientas reales de seguridad informática.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	El proyecto deberá desarrollarse dentro del período establecido para el proyecto, cumpliendo con las etapas de implementación del kernel, desarrollo del programa intermedio, integración del sistema y visualización final en el dashboard.

2. Enunciado de la Práctica

2.1 Descripción del problema a resolver

En los sistemas operativos modernos, la capacidad de monitorear y analizar el comportamiento del sistema en tiempo real es fundamental para garantizar su estabilidad, rendimiento y seguridad. Herramientas tradicionales como *top* o *htop* permiten observar el consumo de recursos, pero no incorporan mecanismos avanzados de detección de amenazas ni análisis de comportamiento a nivel kernel.

Por otro lado, las soluciones antivirus convencionales operan principalmente en espacio de usuario, lo que limita su capacidad de acceso directo a estructuras internas del sistema operativo y reduce su efectividad para detectar comportamientos anómalos en tiempo real. Además, muchos sistemas carecen de mecanismos integrados que permitan correlacionar métricas del sistema con eventos de seguridad, como modificaciones sospechosas de archivos o ejecución de procesos con patrones inusuales.

Ante esta problemática, se plantea la necesidad de desarrollar un sistema integral que combine capacidades de monitoreo de recursos, análisis de procesos y detección de amenazas, utilizando una arquitectura que permita la interacción directa con el kernel mediante llamadas al sistema personalizadas (syscalls).

El problema a resolver consiste en diseñar e implementar un sistema capaz de:

- Recolectar métricas del sistema en tiempo real desde el kernel, incluyendo uso de memoria, estado de páginas y comportamiento de procesos.
- Analizar el comportamiento de los procesos para identificar patrones anómalos en el consumo de recursos.
- Detectar archivos potencialmente maliciosos mediante un mecanismo de firmas basado en hashes (hash blacklist), simulando el funcionamiento de un motor antivirus.

- Identificar cambios en archivos del sistema mediante comparación de hashes y timestamps, permitiendo detectar modificaciones no esperadas.
- Generar alertas de seguridad clasificadas por nivel de severidad (LOW, MEDIUM, HIGH), en función de las condiciones detectadas.
- Controlar el acceso y las funcionalidades del sistema mediante autenticación de usuarios utilizando PAM, diferenciando entre usuarios con privilegios administrativos y usuarios estándar.

Para lograrlo, el sistema deberá integrar múltiples componentes: un conjunto de syscalls implementadas en el kernel para la obtención de información, un programa intermedio en espacio de usuario encargado de procesar y transmitir los datos, un backend que gestione la lógica de análisis y detección de amenazas, y un dashboard web que permita la visualización de métricas y alertas en tiempo real.

El objetivo es que el estudiante comprenda de manera integral cómo se construyen herramientas de observabilidad y seguridad desde el nivel más bajo del sistema operativo, y cómo estas pueden integrarse en una arquitectura completa que simule el comportamiento de soluciones reales de monitoreo y ciberseguridad.

2.2 Alcance de la proyecto

El sistema a desarrollar consiste en una **plataforma integral de monitoreo y análisis de seguridad a nivel kernel**, compuesta por múltiples módulos interconectados que permiten recolectar, procesar, analizar y visualizar información del sistema en tiempo real.

El desarrollo se divide en los siguientes componentes obligatorios:

2.2.1 Llamadas al sistema personalizadas (Kernel)

El estudiante deberá implementar un conjunto de llamadas al sistema (syscalls) en el kernel de Linux que permitan obtener información del sistema y realizar operaciones de análisis.

A. Métricas del sistema (obligatorio)

`sys_get_system_monitor()`: Debe proporcionar como mínimo las siguientes métricas:

- memoria_usada
- memoria_libre
- memoria_cache
- swap_usada
- fallos_menores (minor page faults)
- fallos_mayores (major page faults)
- paginas_activas
- paginas_inactivas

Adicionalmente debe generar una lista de los 10 procesos que más memoria consumen (top procesos).

Cada proceso debe incluir:

- nombre del proceso
- PID
- porcentaje de uso de memoria

Nota: Esta syscall se implementó en la práctica 6

C. Análisis de archivos (obligatorio)

`sys_file_analize()`: Permite trabajar con archivos del sistema:

- Obtener información de un archivo:
  - tamaño
  - timestamp de última modificación
- Calcular el hash de un archivo (**utlizar SHA-256**)

Esta syscall es clave para:

- detección de cambios
- comparación con blacklist

D. Escaneo de procesos (obligatorio)

`sys_scan_processes()`: Deberá recorrer los procesos activos y permitir detectar un comportamiento anómalo (ej: alto consumo)

Importante:

- La syscall **NO debe clasificar severidad**
- Solo devuelve datos crudos
- Quien calcula la severidad es el **backend**

E. Gestión de archivos sospechosos (simulada)

Se deberán implementar las siguientes syscalls para la gestión de archivos identificados como sospechosos o maliciosos. Estas operaciones serán simuladas, por lo que no es necesario realizar un aislamiento real a nivel de sistema de archivos, sino representar su comportamiento de forma controlada.

Se deberán de implementar las siguientes syscalls:

1. `sys_quarantine_file(path):` Permite marcar un archivo como en cuarentena dentro del sistema.

Funcionalidad:

- Recibe la ruta de un archivo como parámetro.
- Verifica que el archivo exista en el sistema.
- Registra el archivo en una estructura interna (lista o arreglo en kernel o estructura simple) de archivos en cuarentena.
- Evita duplicados (no debe agregarse dos veces el mismo archivo).
- Puede almacenar información básica como**:**
    - ruta del archivo
    - timestamp del momento en que fue puesto en cuarentena

Comportamiento esperado:

- No elimina ni mueve físicamente el archivo.
- Simula el aislamiento marcándolo como "**no confiable**".
- Retorna un código de éxito o error según el resultado de la operación.

2. `sys_restore_file(path):` Permite restaurar un archivo previamente marcado como en cuarentena.

Funcionalidad:

- Recibe la ruta del archivo como parámetro.
- Verifica si el archivo se encuentra en la lista de cuarentena.
- Si existe, lo elimina de dicha lista.

Comportamiento esperado:

- Simula la restauración del archivo a un estado "**seguro**".
- Si el archivo no se encuentra en cuarentena, debe retornar un error controlado.
- No modifica físicamente el archivo en disco.

3. `sys_get_quarantine_list():` Permite obtener la lista de archivos actualmente en cuarentena.

Funcionalidad:

- Retorna una estructura con todos los archivos registrados en cuarentena.
- Cada elemento debe incluir al menos:
    - ruta del archivo
    - timestamp de cuarentena

Comportamiento esperado:

- La información será consumida por el programa en espacio de usuario (daemon).
- Debe permitir recorrer todos los elementos de forma estructurada.
- Puede implementarse como:
    - buffer estructurado
    - arreglo de structs

Nota: Estas syscalls son **simulaciones**, **NO** deben realizar:

    - eliminación de archivos
    - cambios en permisos del sistema
    - movimiento físico de archivos
- La lógica de decisión (cuándo cuarentenar o restaurar) **no pertenece al kernel**, sino al programa en espacio de usuario.
- El kernel únicamente:
    - registra
    - almacena
    - expone la información

F. Syscall de simulación de fallo crítico

`sys_simulate_panic(msg)`: Permite mostrar un mensaje de error en el log del kernel, esta **no** debe de provocar un kernel panic real. Mostrará el mensaje recibido.

Ejemplo:

[SECURITY] Simulated kernel panic triggered

Casos en los que se debe activar:

El código intermedio deberá invocar a sys_simulate_panic(msg) en los siguientes escenarios:

1. Detección de archivo malicioso (hash blacklist)

- Cuando un archivo coincida con una firma catalogada como HIGH
- **Ejemplo:**
    - Simulated.Ransomware.Encryptor
    - Simulated.Backdoor.Listener

2. Múltiples alertas críticas simultáneas

- Cuando existan varias condiciones anómalas al mismo tiempo, **por ejemplo**:
    - alto consumo de memoria
    - alto número de fallos de página
    - proceso sospechoso activo

Esto simula una degradación severa del sistema.

3. Comportamiento anómalo persistente

- Cuando una condición crítica se mantenga durante varios ciclos de monitoreo consecutivos
- **Ejemplo**:
    - proceso consumiendo memoria excesiva durante múltiples intervalos

Objetivo de la simulación

- Representar cómo un sistema podría reaccionar ante condiciones críticas
- Permitir la visualización de eventos extremos en el dashboard
- Reforzar el concepto de severidad en sistemas de monitoreo y seguridad

Restricciones importantes (Kernel)

- Todas las syscalls deben implementarse en C
- Deben registrarse correctamente en el kernel
- No se permite duplicar funcionalidad entre syscalls
- No se permite lógica compleja de decisión Ej: Severidad de exploit (eso va en user space)
- No se permite el uso de `panic()` real del kernel
- No debe afectar la estabilidad del sistema operativo

Adicionalmente se debe reutilizar la syscall existente `sys_get_process_info(pid)` para obtener información detallada de un proceso específico.

2.2.2 Programa intermedio (Daemon en C)

Este componente será el núcleo del sistema en espacio de usuario.

Funciones principales

El programa deberá:

- Invocar periódicamente las syscalls
- Recopilar métricas del sistema
- Analizar procesos
- Escanear archivos del sistema
- Generar estructuras de datos en formato JSON
- Enviar los datos al dashboard (HTTP o WebSocket)

Ejecución

- Debe ejecutarse como un **daemon** (segundo plano)
- Debe permitir ejecución continua
- Debe manejar errores sin finalizar abruptamente

Concurrencia (obligatorio)

El programa deberá implementar **al menos 2 hilos**:

Thread 1 → Monitoreo

- métricas del sistema
- procesos

Thread 2 → Escaneo de archivos

- cálculo de hashes
- comparación con registros

1. Detección basada en firmas (Hash Blacklist)

El sistema deberá implementar un mecanismo de detección basado en firmas:

Requisitos:

- Mantener una base de datos local de hashes maliciosos (**hash_blacklist.json**)
- Un directorio a monitorear (ej: `/home/user/monitor/`)
- Debe contener **8 firmas definidas**
- Cada firma debe incluir:
    - hash
    - nombre
    - severidad (LOW, MEDIUM, HIGH)
    - descripción

Para realizar pruebas del funcionamiento se puede utilizar los archivos que se encuentren en el siguiente enlace: Archivos de prueba hash blacklist

Funcionamiento

Para cada archivo escaneado:

1. Calcular hash
2. Comparar contra la blacklist
3. Determinar si existe coincidencia

Detección de cambios en archivos

El sistema deberá:

- Registrar archivos previamente analizados
- Almacenar:
    - hash
    - timestamp

Debe detectar:

- archivos nuevos
- archivos modificados

2. Sistema de alertas (obligatorio)

El daemon deberá generar alertas en los siguientes casos:

- Alto consumo de memoria
- Page faults elevados
- Procesos sospechosos
- Archivo nuevo detectado
- Archivo modificado
- Coincidencia con hash malicioso

Clasificación de severidad:

Las alertas deberán clasificarse como:

- LOW
- MEDIUM
- HIGH

Ejemplo:

Evento	Severidad
Archivo nuevo	LOW
Alto consumo memoria	MEDIUM
Hash malicioso detectado	HIGH

Nota: La severidad se calcula en el daemon (**NO en kernel**)

Acción ante severidad HIGH

Cuando se detecte una amenaza HIGH:

- El daemon deberá invocar:
  `sys_simulate_panic(msg)`

Gestión de usuarios (PAM + Login)

El sistema deberá incluir autenticación de usuarios según los grupos a los que pertenezca dicho usuario.

Requisitos:

- Implementar login
- Usar PAM para autenticación
- Definir roles:
  - **admin_user** (administrador)
  - **common_user** (usuario común)

El programa debe ser capaz de verificar si un usuario pertenece a uno de estos grupos utilizando el nombre de usuario y contraseña, regresando un mensaje de éxito o fallo.

Si un usuario pertenece a ambos grupos, se tomará como administrador.

Permisos

Acción	Administrador	Usuario
Ver dashboard	✔	✔
Ver alertas	✔	✔
Activar escaneo	✔	X
Desactivar escaneo	✔	X
Obtener información de un proceso con su PID	✔	X

Restricción importante

- El control de permisos debe realizarse en el programa en C.
- No se permite delegar esta lógica al frontend.

2.2.3 Dashboard Web (Frontend)

El sistema deberá incluir una interfaz gráfica accesible mediante navegador web, cuya función será visualizar en tiempo real la información generada por el programa en espacio de usuario (daemon en C), así como presentar alertas de seguridad y permitir la interacción básica del usuario según su rol.

Visualización de métricas (obligatorio)

El dashboard deberá reutilizar las gráficas definidas en la Práctica 6. Estas gráficas deben alimentarse con los datos generados por el daemon en tiempo real.

Panel de alertas (obligatorio)

El dashboard deberá incluir una sección visible de alertas de seguridad generadas por el sistema.

Cada alerta deberá mostrar:

- tipo de evento (memoria, proceso, archivo)
- descripción
- nivel de severidad (LOW, MEDIUM, HIGH)
- fecha y hora del evento

Las alertas deben mostrarse de forma ordenada (más recientes primero).

Indicadores de severidad

Las alertas deberán diferenciarse visualmente según su severidad:

- LOW → informativo
- MEDIUM → advertencia
- HIGH → crítico

Puede utilizarse color u otro mecanismo visual para diferenciarlas.

Visualización de archivos analizados

El dashboard deberá incluir una sección donde se muestre:

- lista de archivos analizados
- estado del archivo:
    - limpio
    - modificado
    - sospechoso
- hash actual (opcional mostrar parcial)
- timestamp de última modificación

Visualización de amenazas detectadas

Cuando se detecte un hash malicioso, el dashboard deberá mostrar:

- nombre de la amenaza (firma)
- archivo afectado
- severidad
- descripción

Sistema de autenticación (login)

El dashboard deberá incluir un sistema de inicio de sesión que permite autenticarse contra el sistema (PAM).

El usuario deberá ingresar:

- nombre de usuario
- contraseña

Si el usuario no pertenece a ninguno de los grupos (common_user, admin_user) no deberá poder acceder al sistema.

Control por roles

El dashboard deberá adaptar su comportamiento según el tipo de usuario autenticado:

Usuario administrador:

- Puede visualizar toda la información
- Puede activar/desactivar el escaneo continuo del sistema
- Puede ingresar un PID para visualizar la información del proceso.

Usuario normal:

- Puede visualizar métricas y alertas.

- **No** puede modificar el estado del sistema.
- **No** puede visualizar información de un proceso en específico.

Control de escaneo

El dashboard deberá incluir una opción (botón o control) para:

- Activar escaneo continuo
- Desactivar escaneo continuo

Esta opción solo debe estar disponible para usuarios administradores.

Actualización de datos

El dashboard deberá actualizar la información de forma periódica (intervalos definidos), o mediante comunicación en tiempo real (WebSockets)

Consideraciones técnicas

- El dashboard debe desarrollarse con HTML, CSS y JavaScript
- No es obligatorio el uso de frameworks, es opcional.
- Debe consumir los datos generados por el programa en C (daemon)

Nota importante (para evitar errores comunes)

- El dashboard **no realiza análisis ni detección**
- Solo visualiza la información procesada por el daemon
- La lógica de seguridad y validación de permisos debe estar en el programa en C

2.3 Requerimientos técnicos

1. El sistema deberá desarrollarse sobre Linux, trabajando directamente con el kernel.
2. Las syscalls deberán implementarse en C e integrarse correctamente en el kernel.
3. El kernel deberá compilarse y ejecutarse con las syscalls implementadas.
4. El daemon deberá desarrollarse en C y ejecutarse de forma continua.
5. El daemon deberá implementar concurrencia con al menos dos hilos:
    a. monitoreo
    b. escaneo de archivos
6. La comunicación entre el daemon y el dashboard deberá realizarse mediante HTTP o WebSockets, utilizando formato JSON.
7. El sistema deberá implementar autenticación mediante **PAM**.
8. El control de roles y permisos deberá gestionarse en el daemon.

9. El sistema deberá utilizar el archivo `hash_blacklist.json` como base de firmas.
10. El cálculo de hashes deberá realizarse con el algoritmo criptográfico SHA-256.
11. El sistema deberá manejar errores sin finalizar abruptamente.
12. El dashboard deberá desarrollarse con tecnologías web (HTML, CSS, JavaScript).
13. Se permite el uso de frameworks (siempre que se documente el porqué de su uso).
14. No se permite lógica de detección o severidad en el kernel.

3. Entregables

Tipo	Descripción
Repositorio GitLab	Link del repositorio y rama donde se subió el código del kernel modificado, las syscalls implementadas, el programa intermedio y el dashboard web. Todo el código realizado para el proyecto debe de ir dentro de la carpeta **ProyectoUnico.**
Modificación llamadas al kernel	Código modificado del kernel que incluya la implementación de todas las syscalls solicitadas. Debe seguir la estructura original del kernel e incluir únicamente los archivos modificados. Todo debe estar dentro de la carpeta **Kernel**
Programa intermedio	Código en C con ejecución en segundo plano (daemon) que invoque las syscalls, procese la información, realice el análisis (detección, alertas, severidad) y envíe los datos al frontend en formato JSON mediante HTTP o WebSockets. Debe implementar concurrencia (mínimo 2 hilos) e incluir el ejecutable compilado. Todo dentro de la carpeta **ProgramaIntermedio**.
Dashboard web (frontend)	Código de interfaz gráfica que permite visualizar el estado del sistema, visualizar gráficas, visualizar alertas y escaneo de archivos. Todo dentro de la carpeta **Dashboard.**
Manual Técnico	Documento Markdown que explique las syscalls creadas, funcionalidad del programa intermedio (manejo de errores e hilos) y conexión de esta con el frontend.
Manual de Usuario	Documento Markdown con instrucciones del uso del dashboard web (frontend) tanto para administrador como usuario.

4. Material de apoyo

- Documentación oficial del kernel de Linux: https://docs.kernel.org
- Curso MIT 6.828 – Operating System Engineering (Materiales abiertos).
- Manual man top, man proc, man 2 syscall.

5. Recursos y herramientas a utilizar

Software:

- GNU/Linux, distribución basada en Debian (entorno base).
- QEMU (para ejecución del kernel usax).
- GCC, Make, Vim o VSCode.
- Git y GitHub.

Plataformas:

- GitLab (repositorio y control de versiones).
- UEDI o Classroom (para la entrega final).

Lecturas recomendadas:

1. Capítulos sobre llamadas al sistema y espacio de usuario/kernel de *Understanding the Linux Kernel*.
2. Guías sobre `/proc` y herramientas de monitoreo (`top`, `htop`, `ps`).

6. Cronograma

El cronograma describe las etapas clave de la práctica, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el práctica avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	20/03/2026	20/03/2026
Elaboración	20/03/2026	23/04/2026
Calificación	24/04/2026	27/04/2026

7. Rúbrica de Calificación

7.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	Programa intermedio realizado en el lenguaje C.	
Uso de herramientas requeridas	Uso de kernel personalizado realizado en prácticas anteriores.	
Gestión y entregas de práctica	El repositorio debe contener código modificado del kernel, programa intermedio y dashboard web. Todo dentro de la carpeta llamada **ProyectoUnico**.	
Documentación obligatoria	Se deben entregar los manuales Técnico y de Usuario en formato Markdown.	
Manejo de hilos.	Programa intermedio debe de manejar como mínimo 2 hilos.	
Entrega en Classroom/Uedi	Se debe haber realizado la entrega de la rama del repositorio en la plataforma correspondiente a su tutor. En el caso de Classroom debe haber sido marcado como entregada y no borrador.	

7.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
HABILIDADES		
Implementación de Syscalls (kernel)	30	
Programa en C (daemon: monitoreo, escaneo, alertas, concurrencia)	25	
Dashboard Web (visualización, alertas, control de escaneo)	10	
Gestión de usuarios (PAM + roles + permisos)	10	
Cambio/Funcionalidad adicional (Durante la calificación)	10	
Subtotal HABILIDADES	85	
CONOCIMIENTO		
Manual Técnico	5	
Manual de Usuario	5	
Pregunta del código	5	
Subtotal CONOCIMIENTO	15	
TOTAL GENERAL	**100**	

7.3 Comentarios Generales

_____

_____

_____

_____

No.	Criterio de Evaluación	Punteo Máximo	Satisfactorio (100–61%)	Necesita Mejorar (60–0%)	Punteo Obtenido
Área de Habilidades					
1	Implementación de Syscalls (kernel)	30	Todas las syscalls funcionan correctamente, están bien integradas en el kernel y cumplen con los requerimientos definidos. Rango: 19–30 pts	Syscalls incompletas, errores en implementación o no funcionan correctamente. Rango: 0–18 pts	
2	Programa en C (daemon: monitoreo, escaneo, alertas, concurrencia)	25	El daemon funciona correctamente, usa hilos, realiza monitoreo, escaneo, genera alertas y maneja errores. Rango: 16–25 pts	El daemon presenta fallos, no implementa concurrencia o el análisis es incompleto. Rango: 0–15 pts	
3	Dashboard Web (visualización, alertas, control de escaneo)	10	Visualiza correctamente métricas, alertas y permite control de escaneo según rol. Rango: 6–10 pts	Visualización incompleta, errores en datos o no hay interacción funcional. Rango: 0–5 pts	
4	Gestión de usuarios (PAM + roles + permisos)	10	Login funcional con PAM, roles correctamente implementados y restricciones aplicadas. Rango: 6–10 pts	Datos erróneos, incompletos o syscall falla en devolver la información. Rango: 0–7 pts	
6	Cambio / Funcionalidad adicional	10	Implementa una mejora relevante, bien integrada y funcional durante la evaluación. Rango: 6–10 pts	Cambio incompleto o no funcional. Rango: 0–5 pts	
Sub-total		85			
Área de Conocimiento					
7	Redacción del Manual Técnico	5	El manual técnico está bien estructurado, claro y detallado sobre la implementación de syscalls. Rango: 4–5 pts	Manual poco claro, incompleto o con errores importantes en la explicación técnica. Rango: 0–3 pts	
8	Elaboración del Manual de Usuario	5	Manual de usuario claro, fácil de seguir, con instrucciones correctas para ejecutar el programa de monitoreo. Rango: 4–5 pts	Manual confuso, instrucciones incompletas o incorrectas, difícil de usar para el usuario final. Rango: 0–3 pts	

9	Preguntas sobre el código (defensa)	5	Responde correctamente sobre la implementación, demuestra dominio del código. Rango: 3–5 pts	No logra explicar su código o presenta inconsistencias. Rango: 0–2 pts	
Sub-total		15			
Total		100			